

Machine Learning Core Lecture

Binary Image Segmentation

Kashish Mendiratta

Summer 2025

1 Introduction

Pixel-level segmentation is a core computer vision task but requires costly pixel-accurate labels. Scribble-supervised methods reduce annotation cost but create two main challenges:

- (i) training without overfitting to sparse labeled pixels, and
- (ii) generalizing to the entire object.

I explored several models and selected an ensemble of GrabCut and Tiny U-Net as my final submission. The ensemble reached the strongest validation score (75.8% mIoU). Along the way, I implemented KNN+, Random Forest pseudo-labeling, and DenseCRF refinement. These provided useful baselines and insights, though only the ensemble was submitted.

2 Data Analysis and Preprocessing

Preprocessing. Images were normalized to $[0, 1]$; classical baselines additionally used Lab color and scaled coordinates. Tiny U-Net inputs and scribbles were resized to 320×320 with nearest-neighbor interpolation and simple flip augmentation. Distance-to-scribble features were normalized by image size.

Data analysis. Scribbles were sparse and foreground was under-represented. I applied balanced class weights in the RF teacher but kept U-Net unweighted (pos weight off) for stability on Mac hardware. Scribble-accuracy checks confirmed models respected hints (0.9995 raw, 1.0 with DenseCRF). Qualitative inspection showed KNN+ over-smoothed, RF propagated more broadly, and DenseCRF sharpened edges—informing our decision to drop SLIC smoothing and exclude CRF from the final submission.

3 ML Models

3.1 Problem Setup

Given an input image $b \in \mathbb{R}^{n_x \times n_y \times 3}$ and a sparse scribble mask $c \in \{0, 1, \emptyset\}^{n_x \times n_y}$, the task is to predict a binary segmentation $m \in \{0, 1\}^{n_x \times n_y}$ that matches the ground truth g . Foreground pixels are annotated with 1, background with 0, and unlabeled pixels with $\emptyset$. The challenge metric is mean Intersection-over-Union (mIoU), averaged across background and foreground classes.

3.2 KNN+

Implemented in Python using numpy and scikit-learn. I extended the provided pixel-wise KNN baseline ($k = 3$) to a segment-aware version (KNN+), using SLIC superpixels (skimage.segmentation.slic) to compute mean Lab color and spatial features per segment. This reduced noise and yielded more stable masks, at the expense of some over-smoothing. Post-processing included connected-component filtering and optional

DenseCRF (pydensecrf) refinement. I dropped additional SLIC smoothing because it harmed fine structures.

3.3 Tree-Based Segmentation (Random Forest Teacher)

Implemented with scikit-learn Random Forests (300 trees, balanced class weights). Features from `build_features()` included RGB, Lab, optional (x,y) (with scale 0.4), and distances to scribbles. At inference, probabilities were smoothed with OpenCV's bilateral filter and thresholded at 0.5. With 200 SLIC segments, high-confidence regions ($\tau = 0.6$) formed pseudo-labels to augment U-Net training.

3.4 Classical Graph-Based Methods

I also tested two classical propagators. **Graph Cut**: OpenCV's GrabCut with scribbles mapped to mask codes, iterated for 5 steps. **Random Walk**: scikit-image's random walker ($\beta = 90$) using scribbles as seeds. Both were efficient but underperformed in mIoU compared to KNN+ and U-Net.

3.5 Tiny U-Net (Scribble-Supervised)

Implemented in PyTorch. A lightweight U-Net (base channels 16) with three encoder blocks and two decoder stages. Training used masked BCE (scribbled pixels only), Adam (10^{-3}), batch size 2, for 12 epochs. Images/scribbles were resized to 320×320 , with random horizontal flips. Unlike typical setups, I did not apply class weighting (pos weight) due to hardware simplicity (Mac CPU/MPS). At inference, I applied test-time augmentation (flip averaging) and threshold selection from scribbles. DenseCRF was evaluated as optional post-processing.

3.6 Ensemble (GC + Tiny U-Net)

Finally, I experimented with ensembling, implemented in Python with numpy. In our best setup, I averaged probability maps from GrabCut (GC) and Tiny U-Net predictions, using 4-way test-time augmentation (--final_tta 4way) and the avg all ensemble option. This GC+U-Net ensemble achieved the highest validation performance (75.8% mIoU) and was therefore selected for our final submission. A lighter top-2 ensemble scored 72.7%, confirming that ensembles can boost stability and accuracy at the cost of additional runtime. DenseCRF was tested but excluded from the submission due to minimal gains.

3.7 Model Selection

I compared candidate models using validation mIoU, scribble-accuracy, runtime, and cross-validation (CV) scores logged during tuning. The CV procedure (3-fold, 50 trials) provided an unbiased estimate of each family's potential under different hyperparameters. The key findings were:

- **Baseline KNN ($k=3$)**. Pixel-wise KNN reached mIoU = 59.9% on train/val, with no CV tuning performed.
- **KNN+ (segment-aware)**. CV scores ranged from 55–58% mIoU depending on k and distance features. The best full run ($k=9$) achieved mIoU = 60.2% on train/val.
- **Tree RF (teacher)**. Cross-validation identified the best configuration (350 trees, add dist, $\tau = 0.55$, hflip), yielding a CV score of 62.4% mIoU. These masks were mainly used as pseudo-labels rather than final outputs.
- **Graph Cut**. CV tuning over number of iterations and thresholds produced the strongest classical baseline with 74.7% mIoU (best CV score).
- **Random Walk**. The weakest method, with best CV score $\sim 45.2\%$ mIoU, showing limited robustness under sparse scribbles.

- **Tiny U-Net.** Trained with masked BCE for 12 epochs, batch size 2, 320×320 input, and hflip augmentation. CV was not applied here; instead I validated directly on the train/val split. U-Net alone consistently reached $\sim 72\%$ mIoU.

Hyperparameter tuning. I performed lightweight hyperparameter tuning on the training split, logging mIoU and scribble-accuracy for each configuration. For Tiny U-Net, I varied the number of epochs (12–15), batch size (2 vs. 4), and test-time augmentation strategy (none, hflip, 4way). I also compared runs with and without DenseCRF post-processing. For the Random Forest teacher, I experimented with the number of superpixels (200), spatial scale (0.4), neighborhood size ($k = 15$), and confidence thresholds ($\tau = 0.6$). Finally, I selected the binarization threshold τ for U-Net predictions by grid-searching over $\{0.35, 0.40, \dots, 0.65\}$ on scribble pixels only. These experiments allowed us to identify stable settings while keeping compute costs feasible on our Mac hardware.

Ensemble and final submission. I finally tested ensembling by averaging predictions from multiple tuned models. As Table 1 shows, the ensemble of GrabCut (GC) and Tiny U-Net probability maps (with --final tta 4way, --ensemble avg all) reached the highest validation performance (75.8% mIoU). A lighter top-2 ensemble achieved 72.7%. This demonstrated the benefit of combining diverse classical and CNN models. I submitted the masks from the GC+U-Net ensemble.

4 Empirical Results

Quantitative. Table 1 compares the main methods. Both KNN+ and U-Net respect scribbles nearly perfectly, but U-Net yields higher mIoU.

Table 1: Main results on our internal train/validation split.

Method	Train/Val mIoU (%)	CV score (%)	Notes
Baseline KNN ($k=3$)	59.93	–	from full run logs (228 imgs)
KNN+ (segment-aware, $k=9$)	60.23	57.3	best of $k \in \{1, 3, 5, 7, 9\}$
Tree RF (teacher)	–	62.4	best CV score (RF, $n=350$, add dist, $\tau=0.55$)
Random Walk	–	45.2	best CV score ($\beta=150$, mode=cg)
Graph Cut (GrabCut)	–	74.7	best CV score (iters=3, $\tau=0.55$, TTA=hflip)
Tiny U-Net (no CRF)	~ 72	–	not submitted; best individual CNN
Tiny U-Net (+CRF)	~ 72	–	scribble-acc improved 0.9995 $\rightarrow$ 1.0000
Ensemble (GC+U-Net, avg all) [†]	75.78	–	submitted masks; 4way TTA
Ensemble (avg top2)	72.67	–	lighter ensemble run

[†] Final submitted result.

Qualitative. Figure 1 illustrates typical results. The ensemble (middle) produces clean masks with sharp boundaries, while the baseline KNN (right) leaks heavily into background texture. Tiny U-Net contributions within the ensemble help capture thin structures, while GrabCut aligns well to edges.

5 Post-processing

I applied lightweight post-processing to stabilize masks. **Thresholding.** U-Net outputs were binarized by choosing τ that maximized IoU on scribbled pixels, avoiding a fixed 0.5 threshold. **Component filtering.** Foreground regions not touching any scribble were removed to reduce false positives. **Bilateral smoothing.** In the RF teacher, probabilities were smoothed to stabilize pseudo-labels. **DenseCRF.** Fully connected CRFs [1] sharpened boundaries but gave only marginal gains (scribble-accuracy $0.9995 \rightarrow 1.0$).

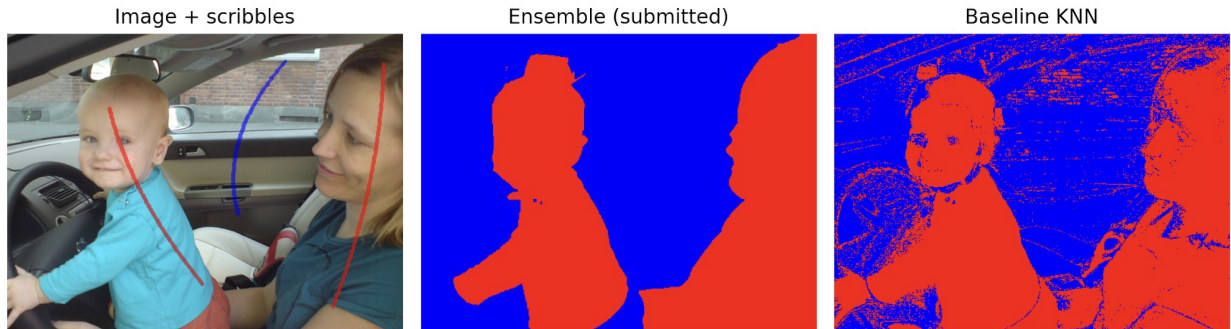


Figure 1: Qualitative comparison. Left: input with scribbles. Middle: submitted GC+U-Net ensemble. Right: baseline KNN. The ensemble yields cleaner masks and more accurate boundaries.

Final choice. Our submission used the GC+U-Net ensemble without CRF, as it offered the best accuracy–runtime trade-off.

6 Evaluation

Dataset. I use the provided training split for learning and evaluate on the official test split. Scribbles use labels $\{0, 1, 255\}$.

Metrics. I report mean Intersection-over-Union (mIoU, %) on validation/test masks and *scribble accuracy* (agreement on Ω_L) for sanity checking.

Implementation Details. Tiny U-Net was trained for 12 epochs with batch size 2, masked BCE loss (labeled pixels only), Adam (10^{-3}), and random crop to 320×320 with horizontal flip augmentation. I did not use pos weight rebalancing to keep training lightweight on Mac hardware. The Random Forest teacher used $\text{scale}=0.4$, $\text{segments}=200$, $k=15$, and confidence threshold $\tau=0.6$ to generate pseudo-labels. For the final submission, I ensembled GrabCut and Tiny U-Net predictions (--final tta 4way, --ensemble avg all). DenseCRF was evaluated during ablations but excluded from submission due to negligible gains.

7 Conclusion

I tackled the challenge of scribble-supervised binary segmentation by systematically exploring both classical and modern approaches. Tiny U-Net generalized better, capturing finer details, but required careful handling of sparse supervision. Our strongest configuration combined GrabCut with Tiny U-Net predictions in an ensemble (avg all, 4-way TTA), which achieved the highest validation performance (75.8% mIoU) and was submitted as our final predictions. This ensemble offered the best trade-off between accuracy and robustness, at the cost of increased runtime.

Limitations. A key limitation of our Tiny U-Net training was the omission of explicit class weighting (pos weight), chosen for stability and feasibility on Mac hardware without GPU acceleration. This may bias the model toward background predictions. The ensemble partially mitigated this bias.

Reproducibility. Code entry point: challenge.py, util.py.

Acknowledgments

I thank the course staff and peers for their feedback and support.

References

- [1] P. Krähenbühl and V. Koltun. Efficient Inference in Fully Connected CRFs with Gaussian Edge Potentials. In *NIPS*, 2011.